Arquitecturas de Software

PANTALLAZO INICIAL

No todas compiten en el mismo nivel

- Cada estilo responde una pregunta distinta: interfaz, código, distribución o ejecución.

INTERFAZ

MVC
MVVM
Cómo fluye la interacción

→

NÚCLEO

Capas
Clean
Hexagonal

→

DISTRIBUCIÓN

Microservicios
Event-Driven
Cómo colaboran partes

→

DATOS + RUNTIME

CQRS
Serverless
Cómo leer, escribir y ejecutar

Ejemplo

Frontend MVVM

↓

Microservicio Pedidos

↓

Internamente usa Clean Architecture

↓

Publica eventos usando Event-Driven

↓

Separa consultas y escrituras con CQRS

↓

Algunas tareas se ejecutan como funciones serverless

MVC y MVVM separan la UI de formas distintas

- Ambos desacoplan la presentación; difieren en quién coordina y cómo se actualiza la vista.

MVC

Controller recibe la acción.
Model concentra datos y reglas.
View renderiza la respuesta.

Ideal: web server-side y CRUD.
Riesgo: "fat controllers".

Casos: GitHub (Rails) y la primera app de Uber.

MVVM

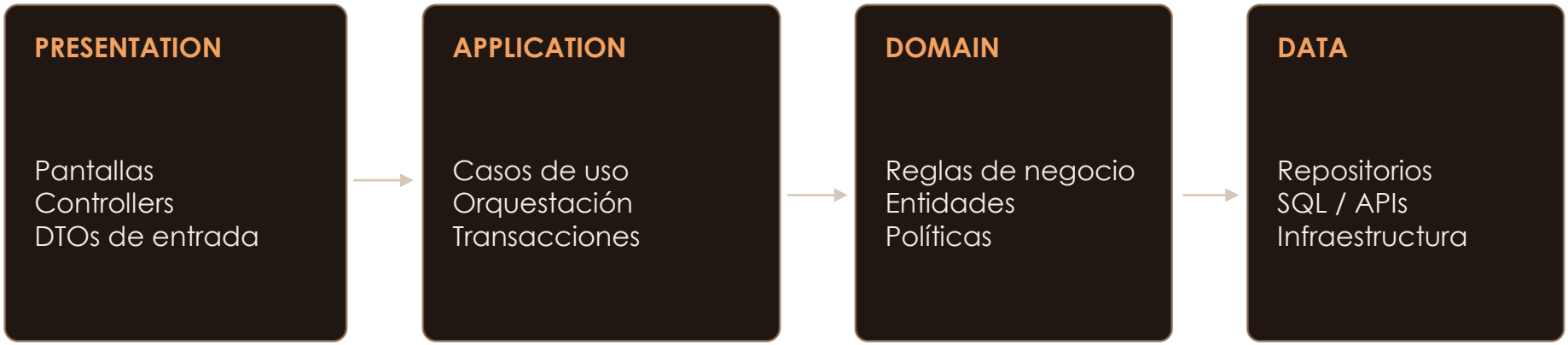
View se enlaza a un ViewModel.
ViewModel expone estado y comandos.
Model mantiene datos y negocio.

Ideal: UI reactiva y XAML (WPF, .NET MAUI).
Riesgo: binding y estado complejos.

Caso: Microsoft Store / WinUI.

Capas ordena el código por responsabilidades

- Es una base común y simple; el riesgo aparece cuando el dominio termina dependiendo de los detalles de datos.



Clean y Hexagonal protegen el dominio

- Comparten inversión de dependencias: los detalles externos dependen del núcleo.

CLEAN ARCHITECTURE

Núcleo al centro
Entidades + casos de uso
Interfaces hacia infraestructura
Dependencias apuntan adentro

Fortaleza: testabilidad.
Caso: Uber DOMA toma ideas de Clean.

HEXAGONAL

Puertos y adaptadores
Puertos = contratos
Adaptadores = DB, API, UI
El dominio ignora tecnología

Fortaleza: cambiar integraciones.
Caso: Netflix Studio Engineering.

DOMINIO PROTEGIDO

Microservicios compran autonomía con complejidad

- Servicios desplegables de forma independiente y alineados a capacidades de negocio.

AUTONOMÍA

Cada equipo evoluciona y escala su servicio sin coordinar un release global.

DOMINIO

Cada servicio necesita propósito, contrato y ownership claros.

DISTRIBUCIÓN

Red, latencia, consistencia eventual y fallos parciales aparecen siempre.

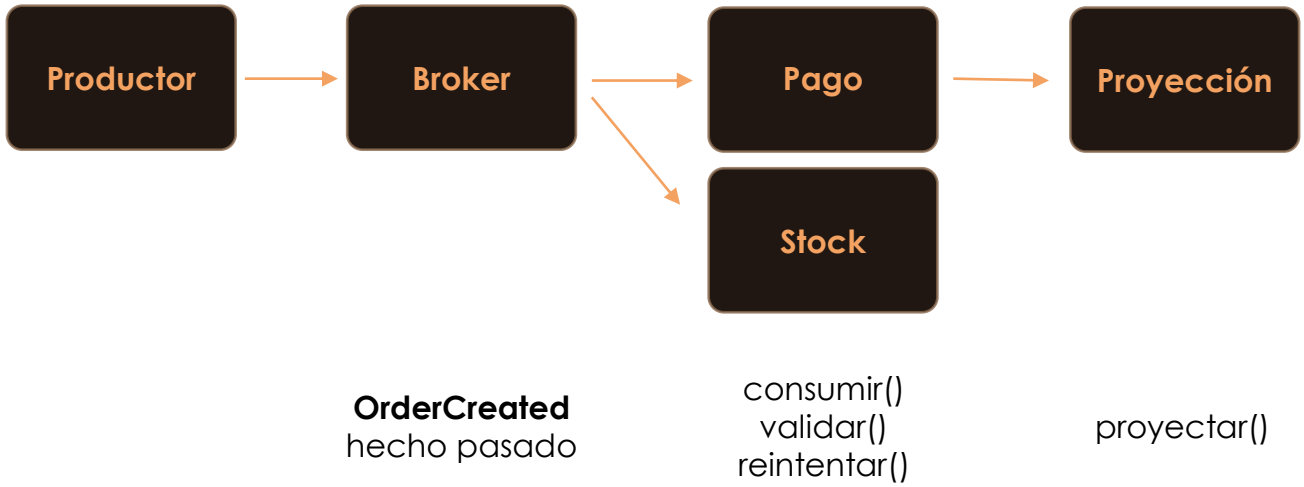
OPERACIÓN

Tracing, automatización, SRE y plataforma dejan de ser opcionales.

```
CASOS: NETFLIX · MERCADO LIBRE · PARAMOUNT+ · UBER
```

Event-Driven desacopla reacciones mediante eventos

- Un productor publica un hecho; el canal lo entrega y varios consumidores reaccionan. Ganás escala y resiliencia, pero debés diseñar idempotencia, orden y evolución de esquemas. Casos: Disney+ y Mercado Libre.



CQRS separa decisiones de escritura y lectura

- Commands cambian estado; Queries devuelven datos. Cada modelo puede optimizarse por separado. Caso público: Juspay.

COMMAND SIDE

- validar reglas de negocio
- cambiar el agregado
- persistir la verdad
- publicar eventos (opcional)

- Optimizado para consistencia

QUERY SIDE

- no cambia estado
- devuelve DTOs o proyecciones
- optimiza joins y filtros
- puede escalar aparte

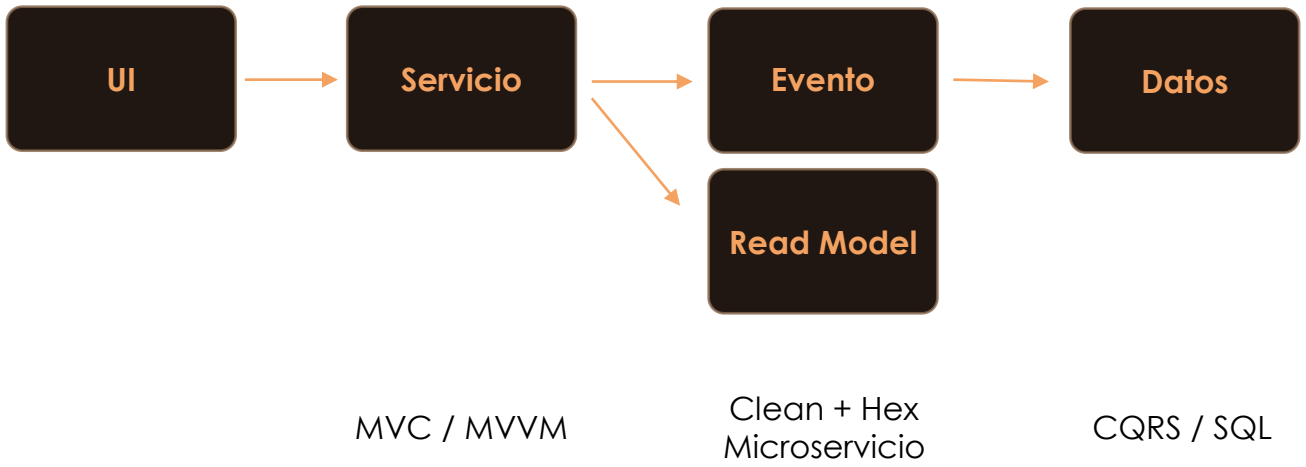
Serverless desplaza la operación al proveedor

- Código y servicios se ejecutan bajo demanda sin administrar servidores directamente. La infraestructura sigue existiendo, pero está gestionada. Casos: LEGO, Coca-Cola Andina y Netflix Cosmos.

1	2	3	4
Evento activa una función.	Escala automática.	Pago por consumo.	Menos control y más lock-in.

En sistemas reales, las arquitecturas se combinan

- Una app puede usar MVVM en el cliente, Clean/Hexagonal dentro de cada servicio, microservicios para desplegar y eventos/CQRS para integrar datos.



Ejemplo

- Imaginemos una tienda:
  - La aplicación móvil utiliza **MVVM** para manejar su pantalla.
  - Llama al **microservicio** de pedidos.
  - Ese **microservicio** utiliza **Clean o Hexagonal** internamente.
  - Al crear un pedido publica **PedidoCreado**.
  - Pagos, Stock y Notificaciones reaccionan mediante **Event-Driven**.
  - Pedidos separa sus comandos y consultas usando **CQRS**.
  - Notificaciones podría ejecutarse como función **Serverless**.

Los casos públicos muestran combinaciones, no recetas

- Las empresas eligen por contexto: escala, organización, riesgo y velocidad de cambio.

NETFLIX

Microservicios, flujos asíncronos y serverless; Hexagonal en Studio Engineering.

MERCADO LIBRE

Miles de microservicios; Fury ofrece messaging y event-sourcing.

PARAMOUNT+

Servicios sobre GKE, Pub/Sub y despliegue multi-región.

DISNEY+ / LEGO

Streaming de eventos a escala; retail serverless para absorber picos.

```
CASOS DE EQUIPOS O PLATAFORMAS CONCRETAS; NO DE TODA LA EMPRESA
```

Elegir arquitectura es elegir un costo

Preguntá primero:

- ¿Dónde está la complejidad real?
- ¿Qué cambia con mayor frecuencia?
- ¿Qué necesita escalar o desplegarse aparte?
- ¿Qué costo operativo puede sostener el equipo?

La arquitectura "más moderna" no siempre es la más adecuada.

SEÑAL	EMPEZÁ POR
UI con estado	MVC o MVVM
Dominio / integraciones	Capas → Clean / Hex
Escala / asimetría / picos	Microservicios + EDA / CQRS / Serverless

Empezá con la complejidad mínima que funcione

- La mejor arquitectura no es la más sofisticada: es la que hace explícitos los trade-offs y puede evolucionar cuando cambian las fuerzas del sistema.

1	2	3	4
¿Qué problema concreto resuelve?	¿Qué cambio quiero aislar?	¿Qué costo operativo agrego?	¿Cómo sabré si funcionó?

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

